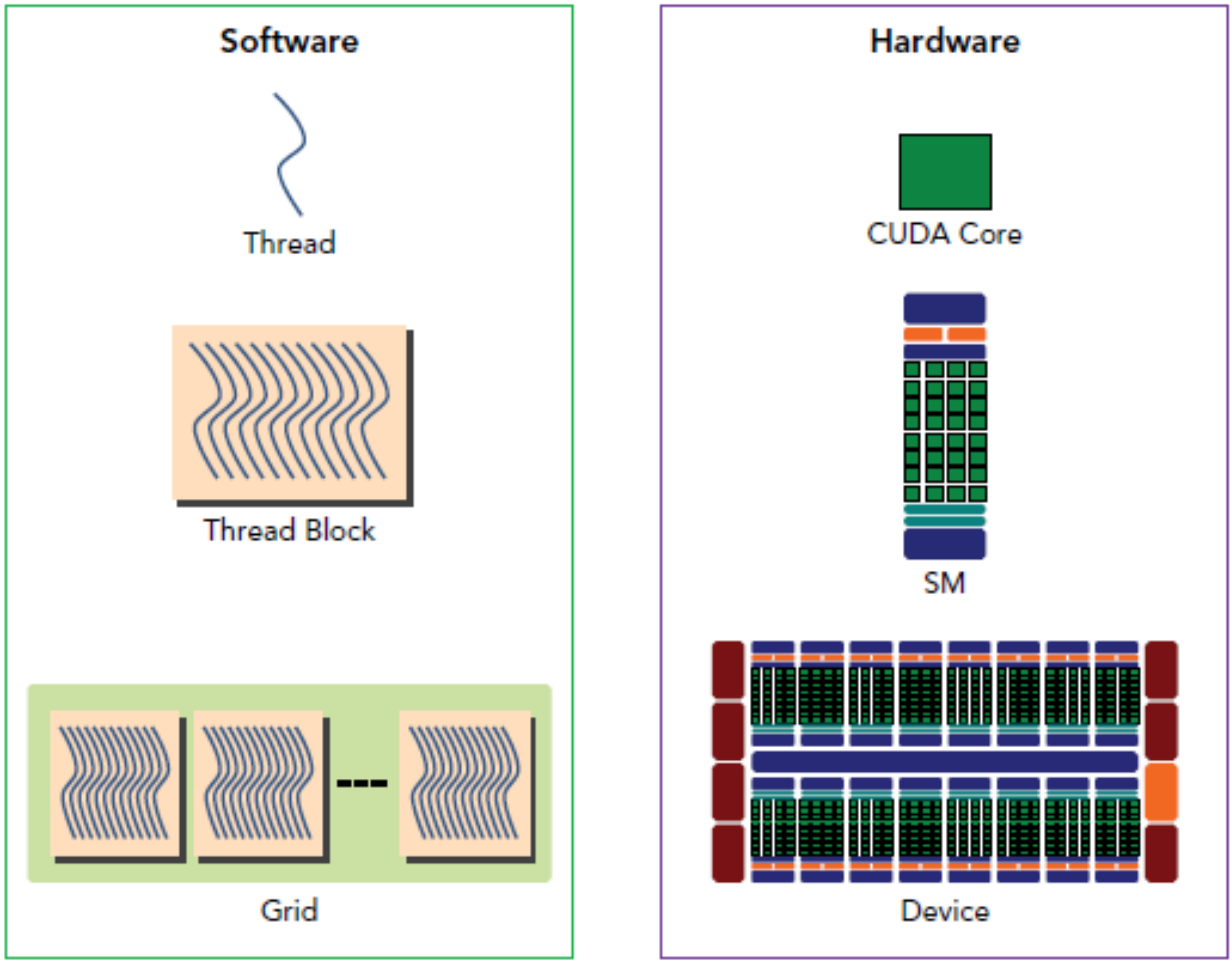
FA4

Part 1:

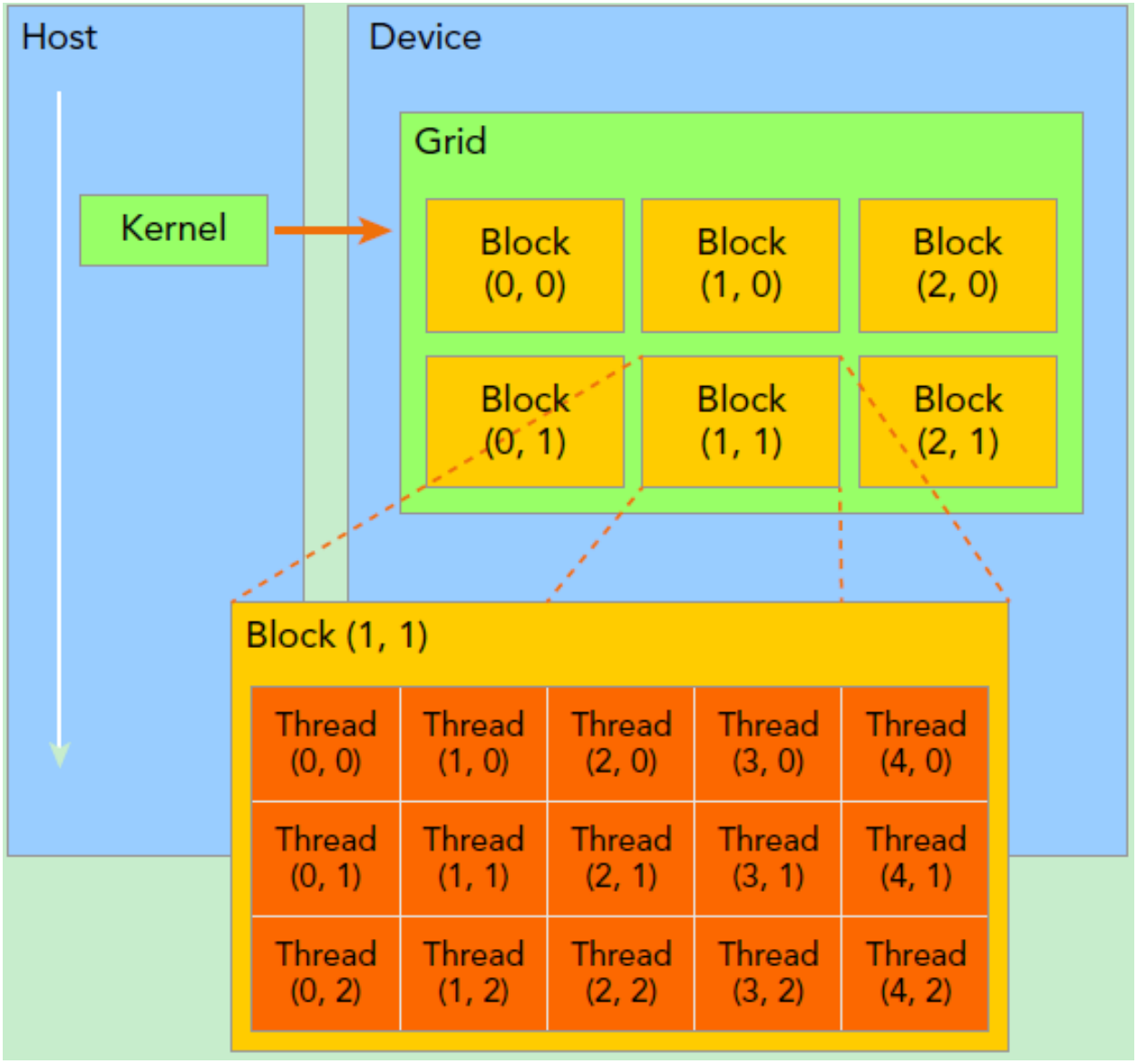
1.
https://modal.com/blog/reverse-engineer-flash-attention-4

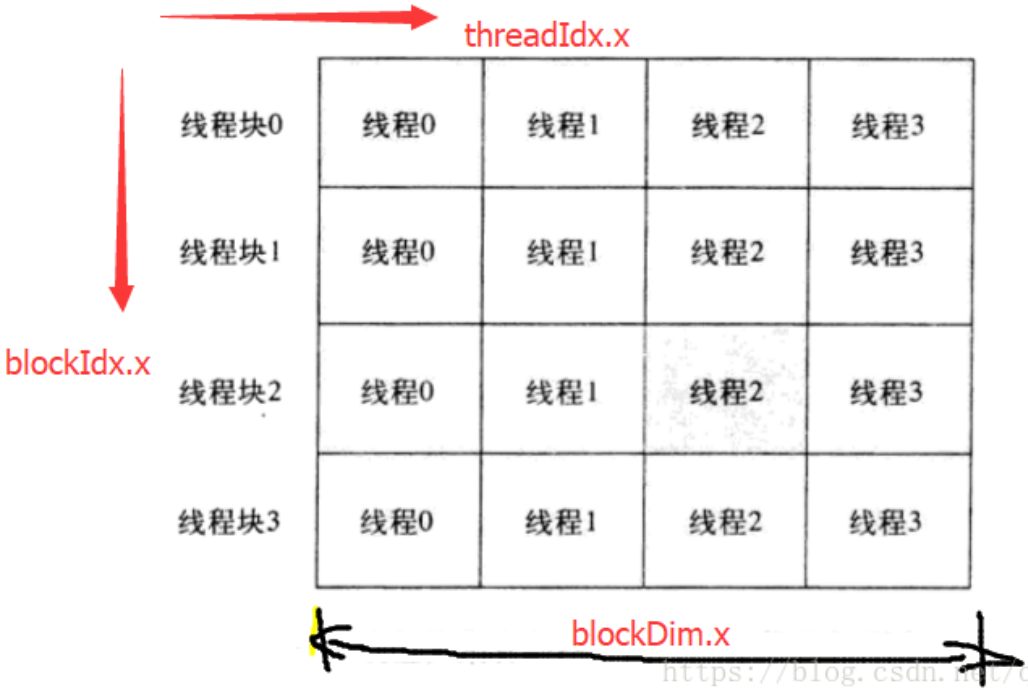
Part 2: GPU architect

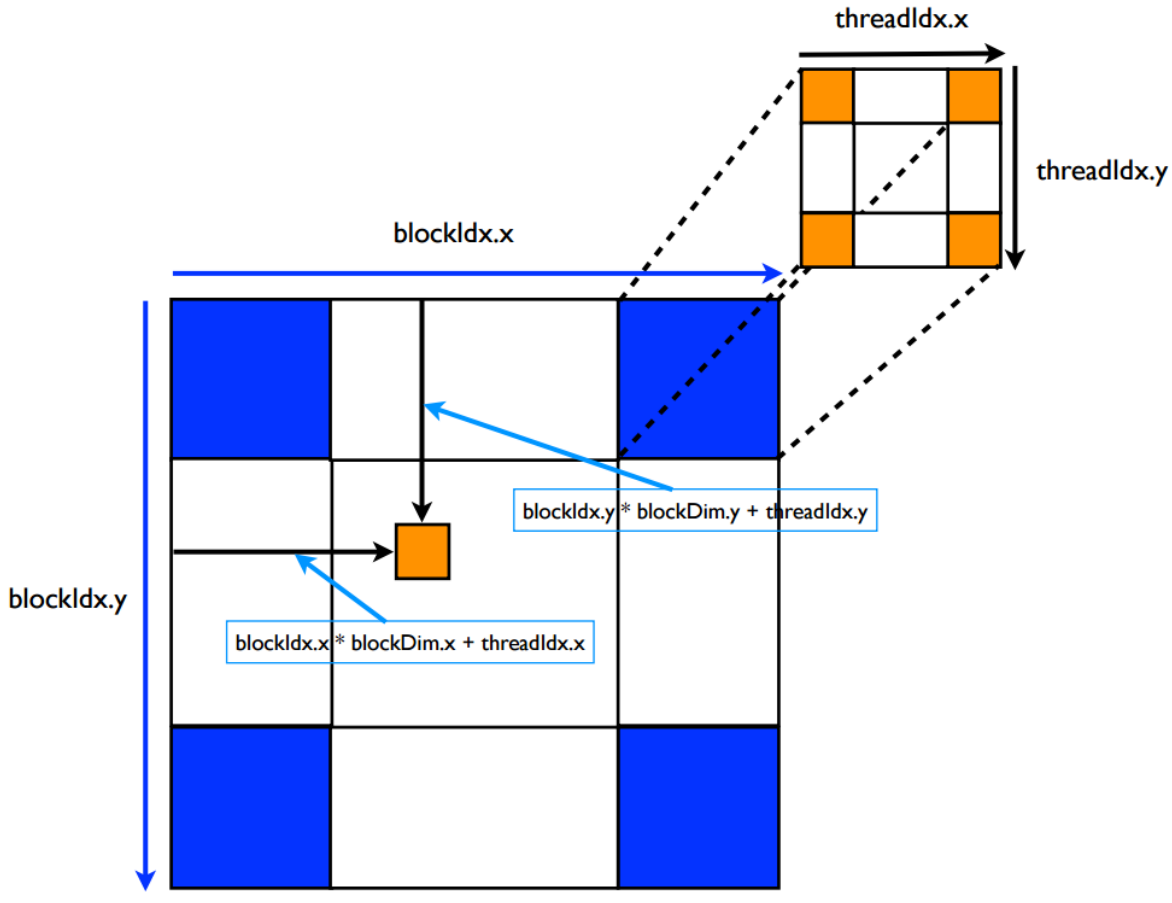
1. CUDA编程——GPU架构，由sp，sm，thread，block，grid，warp说起
https://blog.csdn.net/junparadox/article/details/50540602

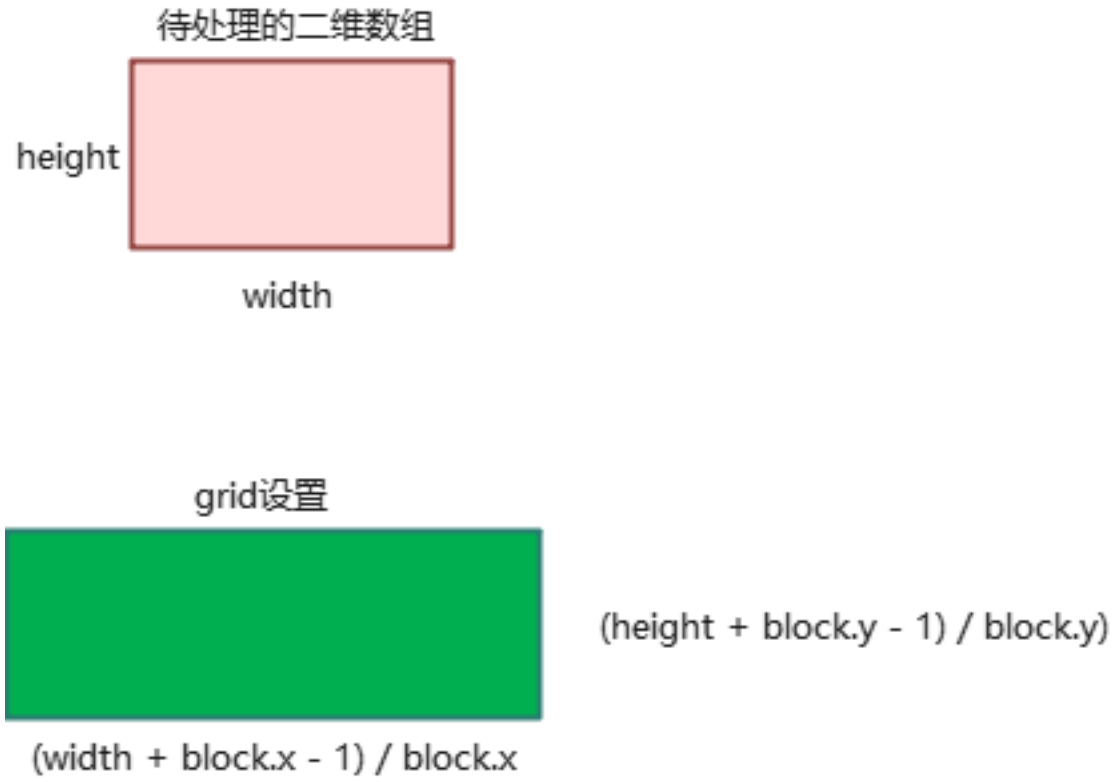


2. Cuda编程中线程块（block）和网格（grid）的结构
https://zhuanlan.zhihu.com/p/721117744





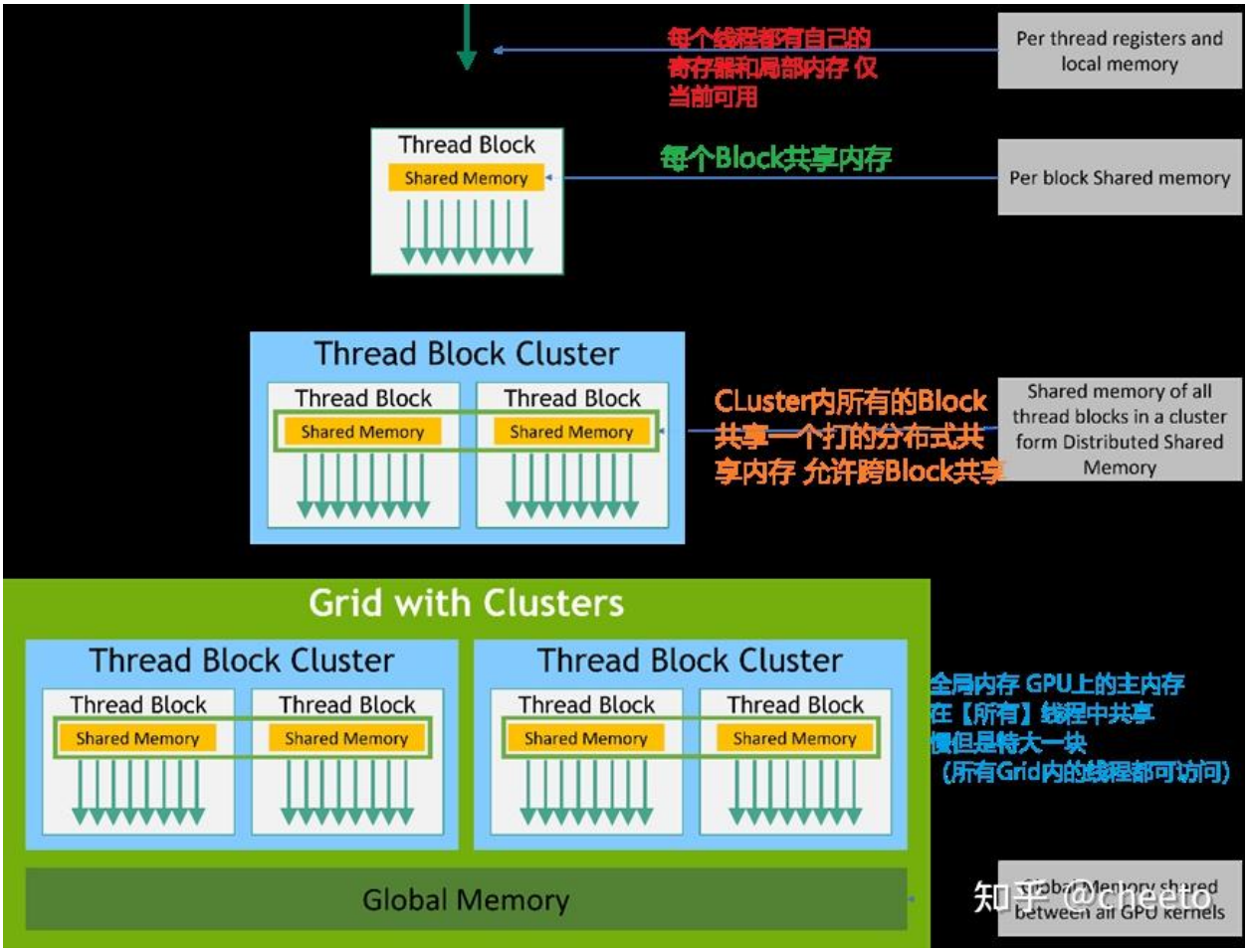


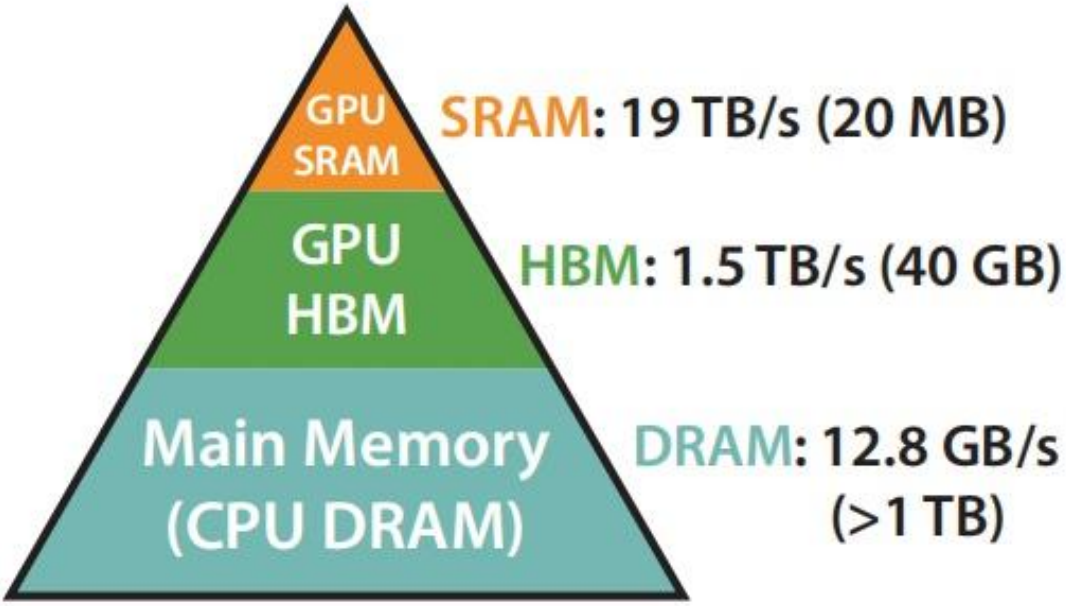


3. 理解CUDA中的thread,block,grid,warp,cluster,CTA,SM,线程之间的区别和关系

https://zhuanlan.zhihu.com/p/8780179505


CTA被放入SM中调度
SM
CTA（Cooperative Thread Array）
Warp
线程


每个线程都有自己的
寄存器和局部内存 仅
当前可用
Per thread registers and local memory
Thread Block
Shared Memory
每个Block共享内存
Per block Shared memory
Thread Block Cluster
Thread Block
Thread Block
Shared Memory
Shared Memory
CLuster内所有的Block
共享一个打的分布式共
享内存 允许跨Block共享
Shared memory of all thread blocks in a cluster form Distributed Shared Memory
Grid with Clusters
Thread Block Cluster
Thread Block Cluster
全局内存 GPU上的主内存
在【所有】线程中共享
慢但是特大一块
(所有Grid内的线程都可访问)
Global Memory
知乎 @cheeto



Memory Hierarchy with Bandwidth & Memory Size

Part 3: Attention, FA1, FA2, FA3

1. Tradition attention, FA1

https://zhuanlan.zhihu.com/p/676655352

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

Algorithm 2 FlashAttention Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}.

1: Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
2: Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil$, $B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$.
3: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
4: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
5: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.
6: **for** $1 \le j \le T_c$ **do**
7: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
8: **for** $1 \le i \le T_r$ **do**
9: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
10: On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
11: On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \mathrm{MASK}(\mathbf{S}_{ij})$.
12: On chip, compute $\tilde{m}_{ij} = \mathrm{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \mathrm{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
13: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
14: On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \mathrm{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
15: Write $\mathbf{O}_i \leftarrow \mathrm{diag}(\ell_i^{\text{new}})^{-1}(\mathrm{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
16: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
17: **end for**
18: **end for**
19: Return $\mathbf{O}, \ell, m, \mathcal{R}$.

Algorithm 4 FLASHATTENTION Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, pseudo-random number generator state $\mathcal{R}$ from the forward pass.

Set the pseudo-random number generator state to $\mathcal{R}$.
Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \ldots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.
Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \ldots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}, \mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ in to T_c blocks $\mathbf{dK}_1, \ldots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \ldots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  Initialize $\mathbf{d\tilde{K}}_j = (0)_{B_c \times d}, \mathbf{d\tilde{V}}_j = (0)_{B_c \times d}$ on SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
    On chip, compute $\mathbf{P}_{ij} = \text{diag}(l_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability p_{drop}.
    On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
    On chip, compute $\mathbf{d\tilde{V}}_j \leftarrow \mathbf{d\tilde{V}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
    On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
    On chip, compute $D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
    On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.
    On chip, compute $\mathbf{d\tilde{K}}_j \leftarrow \mathbf{d\tilde{K}}_j + \tau \mathbf{dS}_{ij}^\top \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
  end for
  Write $\mathbf{dK}_j \leftarrow \mathbf{d\tilde{K}}_j, \mathbf{dV}_j \leftarrow \mathbf{d\tilde{V}}_j$ to HBM.
end for
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

2. FA2
https://zhuanlan.zhihu.com/p/645376942

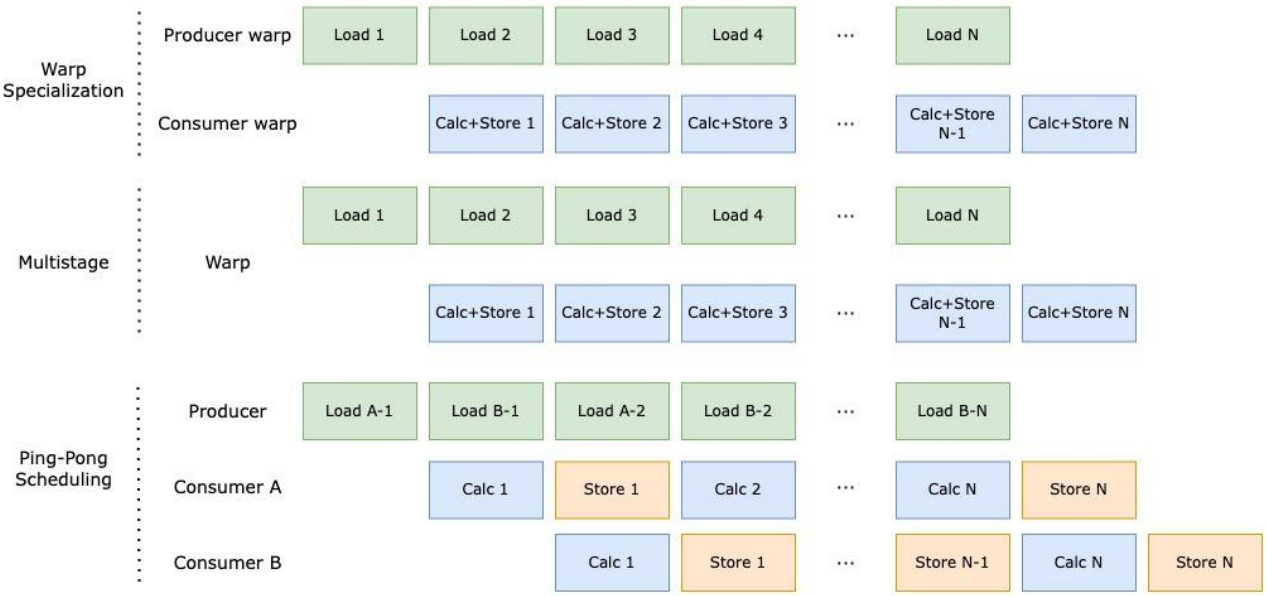
Algorithm 1 FLASHATTENTION-2 forward pass

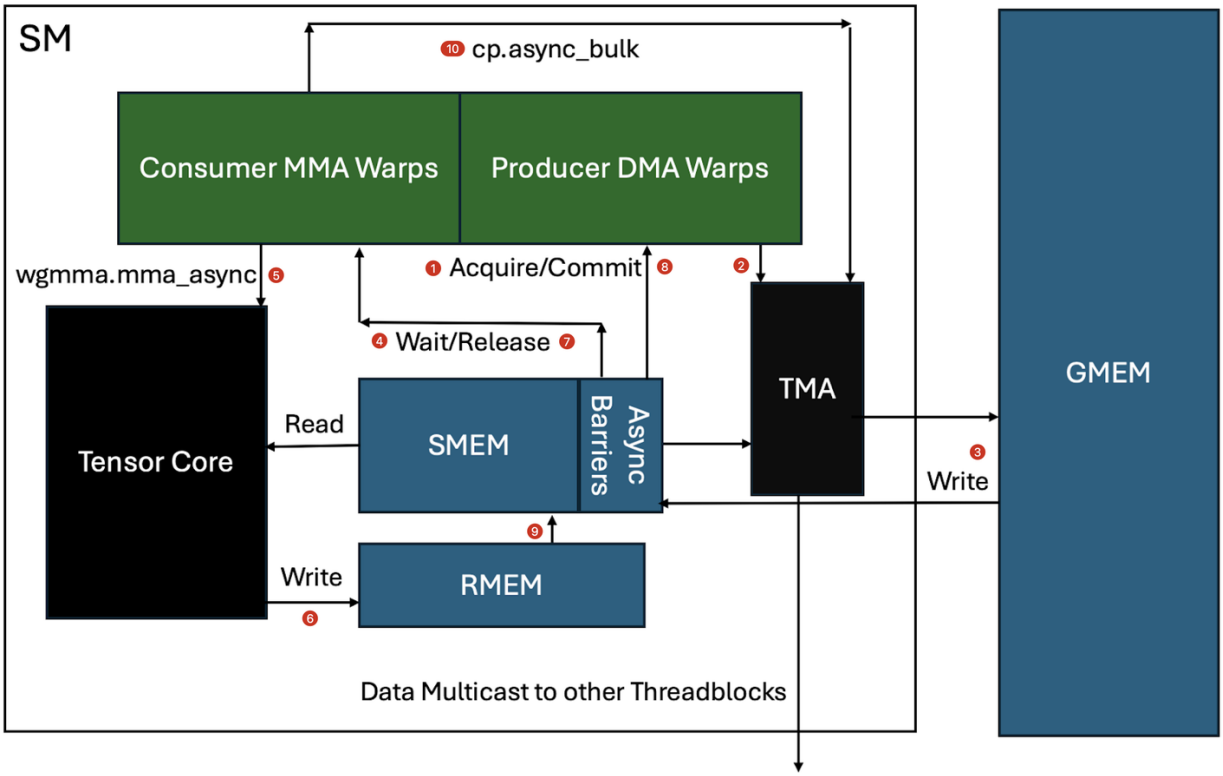
Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.
for $1 \le i \le T_r$ **do**
  Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.
  On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}$, $\ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}$, $m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.
  for $1 \le j \le T_c$ **do**
    Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \text{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{j-1} - m_i^{(j)}} \ell_i^{(j-1)} + \text{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.
    On chip, compute $\mathbf{O}_i^{(j)} = \text{diag}(e^{m_i^{(j-1)} - m_i^{(j)}}) \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.
  end for
  On chip, compute $\mathbf{O}_i = \text{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.
  On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.
  Write $\mathbf{O}_i$ to HBM as the i-th block of $\mathbf{O}$.
  Write L_i to HBM as the i-th block of L.
end for
Return the output $\mathbf{O}$ and the logsumexp L.

3. FA3

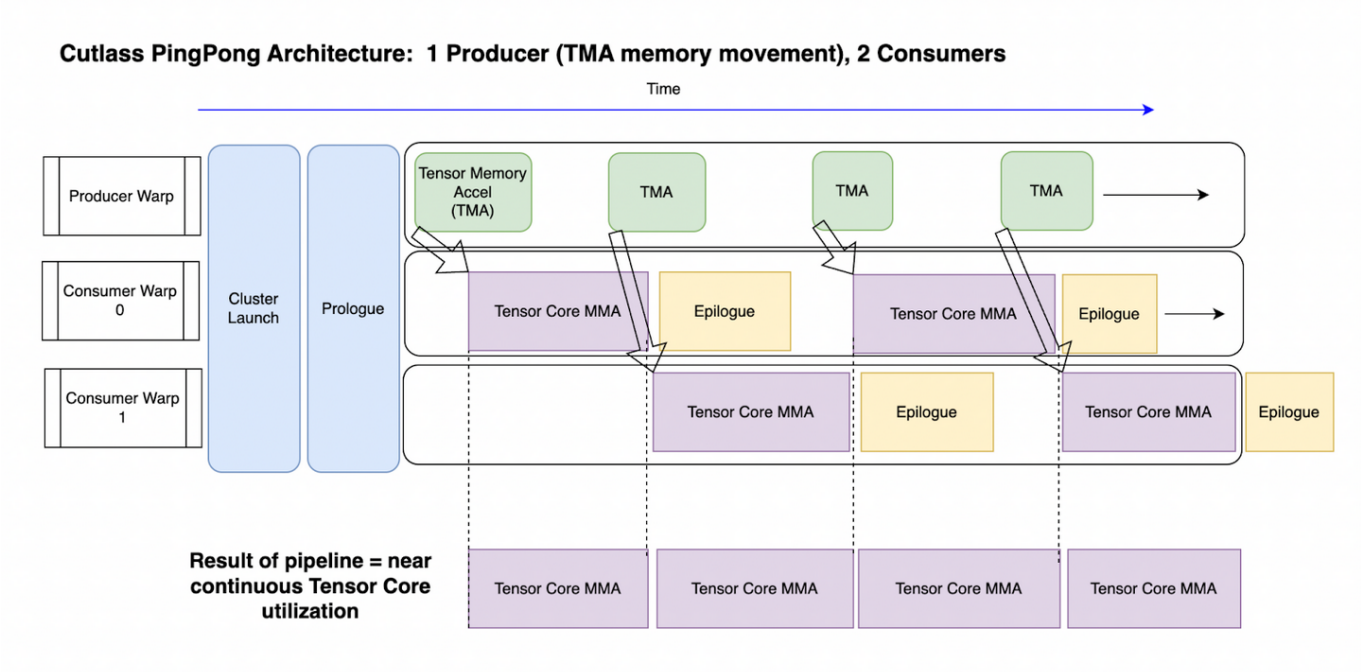
https://zhuanlan.zhihu.com/p/17533058076





1. producer warpgroup 获取 SMEM 缓冲区的 barrier lock。
2. producer warpgroup 通过单个线程向 TMA 芯片发起 TMA 请求。
3. TMA 计算所需的实际 SMEM 地址，将数据移动到 SMEM，并在移动时会进行数据布局转换（如 swizzling），以便在 SMEM 中实现最快速（无 bank conflict）的访问。
   数据也可以 multicast 到其他 SM，或者可能需要等待来自其他 TMA multicast 的数据以完成加载。（thread block cluster 可以在多个 SM 之间共享 SMEM）
4. 此时，barrier 被更新以信号通知数据已到达 SMEM。
5. 相关的 consumer warpgroup 现在开始工作，发出多个 wgmma.mma_async 命令，这些命令将数据从 SMEM 读取到 Tensor Core，随后进行矩阵乘法计算。
6. MMA 累加后的值在完成计算后被写入 RMEM。
7. consumer warpgroup 释放 SMEM 上的 barrier。

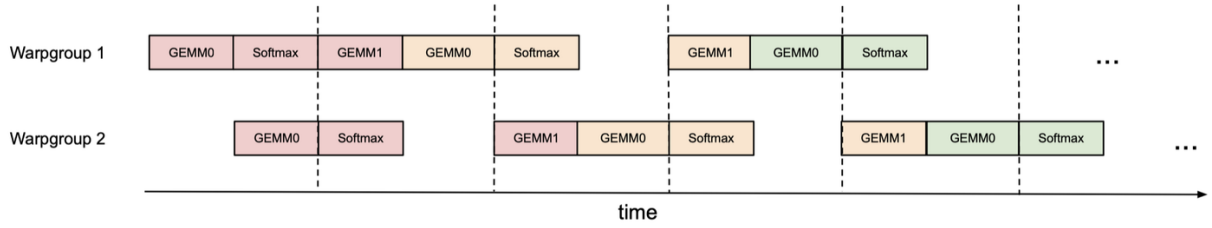
8. producer warpgroup 开始工作，发出下一条 TMA 指令以重新填充现在空闲的 SMEM 缓冲区。
9. consumer warpgroup 同时对累加结果进行后处理（epilogue），然后将数据从 RMEM 移动到不同的 SMEM 缓冲区。
10. consumer warpgroup 发出 cp.async_bulk 命令，将数据从 SMEM 移动到 GMEM。



Producer：warpgroup 中每个线程分配 24 个 registers，主要职责是分发 TMA 指令，由 TMA 将数据从 GMEM 移至 SMEM。数据传输完成后，TMA 会通知相应的 consumer 数据已准备就绪。Producer 会推举出一个 leader 线程发送 TMA 异步指令，指令结束后即停止运行，等待 SMEM buffer 释放；
Consumers：每个 warpgroup 的线程分配 240 个 registers，主要职责是获取 SMEM buffer 的数据，计算 GEMM 和 softmax，释放 buffer 并通知 producer 数据已被释放。随后处理收尾的计算任务、计算结果的数据传输等工作，这也被称为 epilogue 阶段。

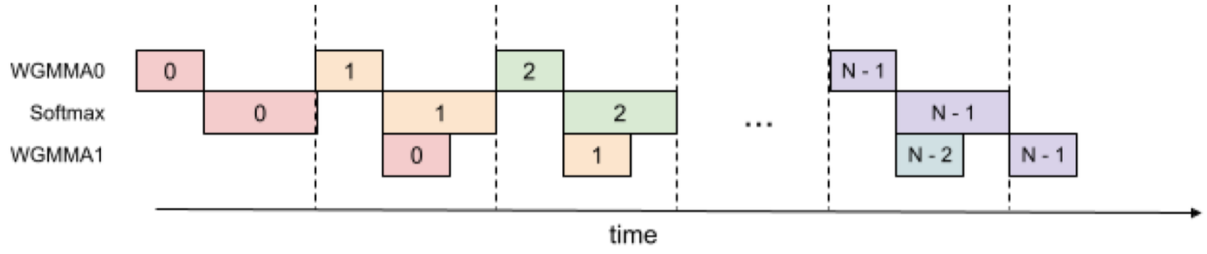
Ping-Pong Scheduling

Ping-pong scheduling 主要发生在两个 consumer warpgroup 之间。由于 WGMMA 的异步性，我们可以同时运行 softmax 和 GEMM 计算，按照下图的调度并用bar.sync在虚线处同步，可以让两个 warpgroup 轮流交替进行 GEMM 计算，以实现更高的 Tensor Core 算力利用率。



Intra-warpgroup Overlapping

在同一个 warpgroup 内部，也可以按照下图编排流水线的方式，实现 GEMM 和 softmax 的计算重叠。下图展示的是 2-stage 流水线方案。



Persistent Kernel

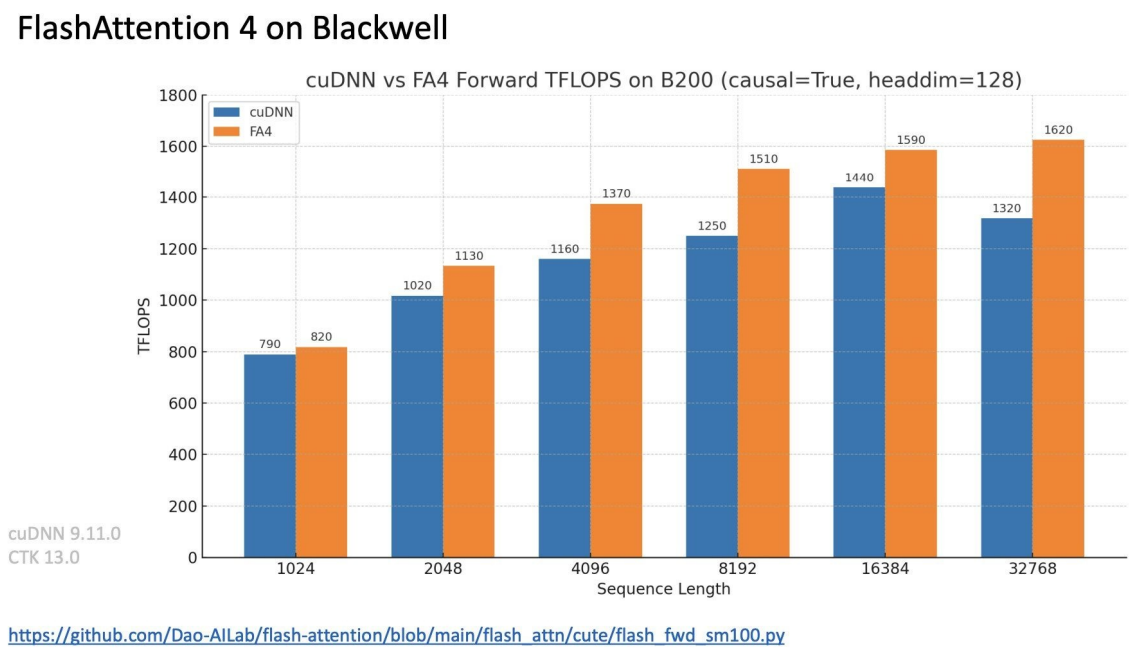
在工程实现方面，FA3 算子在每个 SM 上会启动一个 persistent Kernel，成为一个 persistent thread block。这个 persistent block 在它的生命周期内（一次 kernel launch 的计算中）可以处理多个 thread block 的 tile 分块数据，在两个 thread block 的计算之间，可以将前一个 block 的 kernel prologue 阶段和后一个 block 的 launch 阶段同时进行，由此掩盖了同一 SM 上先后两个 thread block 切换的延迟。

在早期的架构上，在 SM 上并发运行多个 thread block 就能很好地处理延时问题。但在 Hopper 架构下，Tensor Core 的计算已经非常快了，这就要求有更深的流水线来掩盖延时，而更深的流水线阻碍了在一个 SM 上运行多个 thread block，因此 persistent thread block 可以在多个 tiles 和多个 warpgroups 上运行 collective main loops。

Persistent Kernel 是一种宏观上的概念，而 Stream-K 算法是一种 Persistent Kernel 的工程实现，具有负载均衡的特性。

Part 3: FA4

https://modal.com/blog/reverse-engineer-flash-attention-4



https://github.com/Dao-AILab/flash-attention/blob/main/flash_attn/cute/flash_fwd_sm100.py

1. New in Flash Attention 4: this step can use CUDA Cores instead of Special Function Units (SFUs) to perform the exponential step of the normalization. SFUs are intended to provide hardware acceleration for transcendental operations like exponentials. But there are far fewer SFUs than CUDA Cores, which can lead to queueing.

2. When each tile of normalized attention scores is ready, a Correction warp checks if the normalization scaling factor has changed and, if necessary, rescales the final output tile in Tensor Memory (tO).

⚡ New in Flash Attention 4: the choice of when to rescale became much smarter, reportedly cutting down on output rescaling operations by a factor of 10. Roughly: the scaling factor used to be a simple running maximum. Now updates are applied only when the maximum has changed enough to impact numerical stability. This seems like a good, and very portable, idea.